

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY (CHARUSAT)

Faculty of Computer Science & Engineering

Subject: Software Group Project (SGP) — B.Tech CSE

CharusatNeeds

Campus Food Aggregator Platform

Software Requirements Specification (SRS)

Document Version: 1.0 | Date: February 2026

Standard: IEEE 830-1998 / ISO/IEC/IEEE 29148:2018

Project Team

Kush Shah

Team Lead + Full Stack Developer + Security
Tester

Krina Parikh

Frontend Developer

Ishan Shastri

Full Stack Developer

Dhairya Tanna

Backend Developer + Security Architecture

Shrey Raval

System/Security Tester + QA

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions & Acronyms

1.4 References

1.5 Overview

2. Overall Description

2.1 Product Perspective

2.2 Product Functions

2.3 User Classes & Characteristics

2.4 Operating Environment

2.5 Design & Implementation Constraints

2.6 Assumptions & Dependencies

3. System Architecture

4. Specific Requirements

4.1 Functional Requirements

4.2 Non-Functional Requirements

4.3 API Specification

4.4 Database Schema

5. Team & Work Distribution

6. Project Planning & Phases

7. Appendices

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the **CharusatNeeds — Campus Food Aggregator Platform**. It details the functional and non-functional requirements for the system and is intended for the development team, project evaluators, and stakeholders. The document follows the IEEE 830-1998 standard for software requirements specifications.

1.2 Scope

CharusatNeeds is a full-stack web application designed to digitize the campus food ordering experience at CHARUSAT. The system enables students and faculty to browse canteen menus, place orders, track order status in real-time via WebSocket, and manage their profiles. Canteen vendors can manage menus, process orders, offer coupons, and view analytics. The platform is designed for multi-device access across mobile, tablet, desktop, kiosk, and TV displays.

Key Objectives:

- Eliminate physical queues at campus canteens
- Provide real-time order tracking and notifications
- Enable vendors to manage operations digitally
- Ensure enterprise-grade security (OWASP, NIST 800-63B)
- Support responsive deployment across all screen form factors

1.3 Definitions, Acronyms & Abbreviations

Term	Definition
SRS	Software Requirements Specification
JWT	JSON Web Token — Stateless authentication token
STOMP	Simple Text Oriented Messaging Protocol — WebSocket sub-protocol
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
FSSAI	Food Safety and Standards Authority of India
CRUD	Create, Read, Update, Delete operations
OWASP	Open Web Application Security Project

Term	Definition
MFA	Multi-Factor Authentication
RBAC	Role-Based Access Control
REST	Representational State Transfer
WebSocket	Full-duplex communication protocol over TCP
BOGO	Buy One Get One — Coupon type

1.4 References

- IEEE 830-1998: Recommended Practice for Software Requirements Specifications
- ISO/IEC/IEEE 29148:2018: Systems and Software Engineering — Life Cycle Processes — Requirements Engineering
- OWASP Application Security Verification Standard (ASVS) v4.0
- NIST Special Publication 800-63B: Digital Identity Guidelines — Authentication
- Spring Boot 3.x Official Documentation
- React 18 Official Documentation

1.5 Document Overview

Section 2 presents the overall product context and user classes. Section 3 covers the system architecture. Section 4 defines detailed functional and non-functional requirements, API endpoints, and database schema. Section 5 details the team composition and work distribution. Section 6 provides the project planning phases with a Gantt chart. Section 7 contains appendices.

2. Overall Description

2.1 Product Perspective

CharusatNeeds is a standalone, self-contained web application. It does not replace any existing system but introduces a new digital food ordering workflow on the CHARUSAT campus. The system is composed of three primary subsystems:

- **Frontend SPA** — React + TypeScript + Tailwind CSS, served via Vite
- **Backend API Server** — Spring Boot 3.x MVC + JDBC (Java 21)
- **Database** — PostgreSQL 15+

Real-time communication is achieved via STOMP-over-WebSocket (native + SockJS fallback). Email services are handled through SMTP (Brevo/SendinBlue). Authentication supports both local credentials and Google OAuth 2.0.

2.2 Product Functions (High-Level)

Module	Functions	Status
Authentication	User registration, login (local + Google OAuth), JWT token management, refresh tokens, email verification, password reset, CAPTCHA, account lockout, MFA support	Implemented
User Management	Profile CRUD, profile image upload/storage, address management (student/teacher), name validation with email ID retention	Implemented
Canteen Management	Canteen CRUD, open/close toggle, rush hour mode, FSSAI/GST compliance fields, bank details, KYC document upload	Implemented
Menu Management	Menu item CRUD with variants, add-on groups, dietary info, categories, availability scheduling, display ordering, item toggle	Implemented
Shopping Cart	Server-side cart, add/update/remove items, auto-total recalculation, variant and add-on selection, special instructions	Implemented
Order System	Order placement, status lifecycle (Pending→Confirmed→Preparing→Ready→Completed/Cancelled), payment tracking, rejection reasons	Implemented
Real-time Notifications	WebSocket (STOMP) for live order updates, canteen-specific topics, user-specific topics, auto-reconnect	Implemented

Module	Functions	Status
Coupon System	Percentage/flat discounts, BOGO offers, scope (global/category/item), usage limits, validity periods, activation toggle	Implemented
Analytics Dashboard	Revenue stats, order count, top-selling items, mealtime distribution, sales trends (daily/weekly/monthly), review insights	Implemented
Review System	Multi-criteria ratings (food, packing, delivery), comments, anonymous reviews, vendor replies	Implemented
Responsive UI	Mobile-first design with xs/sm/md/lg/xl/kiosk/tv breakpoints, skeleton loading, Framer Motion animations	Implemented
Payment Gateway	UPI/card/wallet integration for online payments	Planned
Push Notifications	Mobile push notifications for order status changes	Planned
Admin Panel	Super-admin dashboard for platform-wide management, vendor approval, analytics aggregation	Planned
Delivery Tracking	Real-time delivery tracking with map integration	Planned

2.3 User Classes and Characteristics

User Class	Description	Access Level
Student (USER)	CHARUSAT students who browse canteen menus, place orders, track status, and write reviews	Browse menus, manage cart, place orders, view order history, manage profile
Faculty (USER)	CHARUSAT faculty/staff with the same ordering capabilities as students, with additional address fields	Same as Student + teacher-specific address fields
Canteen Owner (CANTEEN_OWNER)	Operates a registered canteen on campus. Manages menu, processes orders, runs promotions	Menu CRUD, order management, coupon management, analytics dashboard, profile settings

User Class	Description	Access Level
Administrator (ADMIN)	Platform administrator with full system access	All operations + user management + platform analytics + vendor approval

2.4 Operating Environment

Component	Technology	Version
Frontend Runtime	Node.js	20.x LTS
Frontend Framework	React + TypeScript	18.x + 5.x
CSS Framework	Tailwind CSS	3.x
Build Tool	Vite	6.x
Backend Runtime	Java (JDK)	21
Backend Framework	Spring Boot	3.x
Database	PostgreSQL	15+
WebSocket	STOMP over Native WS + SockJS	—
Email Service	SMTP (Brevo/SendinBlue)	—
Browser Support	Chrome 90+, Firefox 88+, Safari 14+, Edge 90+	—

2.5 Design and Implementation Constraints

- The system must use Spring Boot's JDBC template (not JPA/Hibernate) for database access
- Authentication must be stateless using JWT tokens
- All API endpoints must follow RESTful conventions
- Frontend must be a single-page application (SPA) with client-side routing
- The application must be responsive across 7 breakpoints (xs through tv)
- Security must comply with OWASP ASVS and NIST 800-63B
- Passwords must enforce policy: minimum length, complexity, and history checks

2.6 Assumptions and Dependencies

- Users have access to a modern web browser with JavaScript enabled
- Campus network provides stable internet connectivity

- PostgreSQL database server is available and configured
- SMTP service (Brevo) is accessible for email operations
- Google OAuth client credentials are configured for social login
- The application runs on localhost during development (frontend: 5173, backend: 8080)

3. System Architecture

3.1 High-Level Architecture

The system follows a **3-tier client-server architecture** with a clear separation of concerns:

Layer	Technology	Responsibility
Presentation Layer	React 18 + TypeScript + Tailwind CSS	UI rendering, state management, client-side routing, responsive layouts
Application Layer	Spring Boot 3.x (Java 21)	Business logic, REST API, authentication, WebSocket messaging, email services
Data Layer	PostgreSQL 15+ via JDBC	Data persistence, relational storage, indexing, constraints

3.2 Component Diagram

The application consists of the following major components:

Frontend Components (11 Pages + 25 Components)

Page	Route	Description
Login	/login	User authentication with email/password and Google OAuth
Signup	/signup	New user registration with email verification
Forgot Password	/forgot-password	Password reset request form
Reset Password	/reset-password	Token-validated password reset form
Student Dashboard	/customer/dashboard	Canteen browsing, search, filters, explore section
Customer Menu	/canteen/:id/menu	Menu browsing, item details, add to cart
Customer Profile	/customer/profile	Profile management, address, image upload
Cart Page	/cart	Shopping cart review and modification

Page	Route	Description
Checkout	/checkout	Order placement and payment
Vendor Dashboard	/dashboard	Order management, live notifications, analytics
Menu Management	/canteen/menu	Menu item CRUD, categories, variants, add-ons
Order History	/order-history	Order history with filtering and real-time status
Analytics	/analytics	Sales data, top items, reviews, mealtime distribution
Vendor Profile	/vendor/profile	Vendor profile and canteen settings
Vendor Coupons	/vendor/coupons	Coupon creation and management

Backend Components (11 Controllers + 16 Services)

Controller	Base Path	Endpoints
AuthController	/api/auth	register, login, logout, refresh, google-callback, me, verify-email, resend-verification
UserController	/api/users	profile GET/PUT, address GET/PUT, profile-image upload/GET
CanteenController	/api/canteens	CRUD, open filter, menu CRUD, rush-hour toggle, availability toggle
OrderController	/api/orders	CRUD, my-orders, canteen orders, active/recent orders, status update, cancel, pending count
CartController	/api/cart	get, add, update quantity, remove, clear
VendorController	/api/vendor	add menu item, update item, create/get coupons
CouponController	/api/vendor/coupons	list, create, delete, toggle
AnalyticsController	/api/analytics	dashboard stats, sales data, top items, reviews, mealtime distribution

Controller	Base Path	Endpoints
PasswordResetController	/api/auth	forgot-password, validate-token, reset-password
CaptchaController	/api/captcha	generate, validate
CategoryController	/api/categories	CRUD operations for menu categories

3.3 Security Architecture

Security Layer	Implementation
Authentication	JWT tokens (access + refresh), Google OAuth 2.0, CAPTCHA verification
Authorization	Role-Based Access Control (USER, CANTEEN_OWNER, ADMIN)
Password Policy	Minimum length, complexity rules, password history (prevent reuse), bcrypt hashing
Account Protection	Account lockout after failed attempts, IP-based rate limiting
Session Management	Stateless JWT, refresh token rotation, all-device logout
Input Validation	Server-side validation (Jakarta Bean Validation), input sanitization
Security Headers	X-Content-Type-Options, X-Frame-Options, X-XSS-Protection, Content-Security-Policy
Transport Security	HTTPS enforcement, CORS whitelisting, secure cookie flags
Email Security	Token-based email verification, single-use password reset tokens with expiry
Audit Logging	Security event logging with IP tracking, login attempt recording

4. Specific Requirements

4.1 Functional Requirements

FR-01: User Registration

- FR-01.1: System shall allow users to register with email, password, full name, and mobile number
- FR-01.2: System shall validate email format and reject duplicates
- FR-01.3: System shall enforce password policy (min length, complexity)
- FR-01.4: System shall send email verification link upon registration
- FR-01.5: System shall support Google OAuth 2.0 registration

FR-02: User Authentication

- FR-02.1: System shall authenticate users via email/password with JWT issuance
- FR-02.2: System shall support CAPTCHA verification during login
- FR-02.3: System shall implement account lockout after consecutive failed attempts
- FR-02.4: System shall provide refresh token rotation for session extension
- FR-02.5: System shall support single-device and all-device logout
- FR-02.6: System shall provide secure password reset via email tokens

FR-03: User Profile Management

- FR-03.1: Users shall be able to view and update their profile (name, mobile, gender, DOB)
- FR-03.2: System shall enforce name validation (ID from email must be retained)
- FR-03.3: Users shall be able to upload and view profile images (stored in DB)
- FR-03.4: Students shall manage address (hostel, room, building)
- FR-03.5: Teachers shall manage address (department, staff room)

FR-04: Canteen Browsing

- FR-04.1: System shall display all registered canteens with status, rating, and location
- FR-04.2: Users shall filter canteens by: nearest, great offers, rating 4.0+
- FR-04.3: Users shall search canteens and menu items by keyword
- FR-04.4: System shall display canteen open/closed status in real-time

FR-05: Menu Browsing and Ordering

- FR-05.1: System shall display menu items with name, price, category, availability, dietary info
- FR-05.2: Users shall filter menu by category and search by name/description
- FR-05.3: Users shall add items to cart with variant selection, add-ons, and special instructions

- FR-05.4: System shall maintain server-side cart with auto-total recalculation
- FR-05.5: Users shall place orders with payment method selection

FR-06: Order Lifecycle Management

- FR-06.1: Orders shall follow status lifecycle: Pending → Confirmed → Preparing → Ready → Completed
- FR-06.2: Vendors shall update order status with optional rejection reason
- FR-06.3: Users and vendors shall cancel orders (with reason tracking)
- FR-06.4: System shall broadcast order updates via WebSocket in real-time
- FR-06.5: Users shall view complete order history with filtering

FR-07: Vendor Operations

- FR-07.1: Vendors shall perform full CRUD on menu items (with variants, add-ons, dietary tags)
- FR-07.2: Vendors shall manage menu categories (create, delete)
- FR-07.3: Vendors shall toggle item availability and canteen open/close status
- FR-07.4: Vendors shall create and manage coupons (percentage, flat, BOGO)
- FR-07.5: Vendors shall view analytics: revenue, order count, top items, mealtime distribution
- FR-07.6: Vendors shall manage canteen compliance (FSSAI, GST) and bank details

FR-08: Real-Time Notifications

- FR-08.1: System shall provide WebSocket connection via STOMP protocol
- FR-08.2: New orders shall be broadcast to canteen-specific topics (/topic/canteen/{id}/orders)
- FR-08.3: Order status changes shall be broadcast to user-specific topics (/topic/user/{id}/orders)
- FR-08.4: WebSocket shall auto-reconnect with heartbeat monitoring

FR-09: Review System

- FR-09.1: Users shall submit reviews with overall rating, food/packing/delivery sub-ratings
- FR-09.2: Users shall submit anonymous reviews
- FR-09.3: Vendors shall reply to reviews

4.2 Non-Functional Requirements

ID	Category	Requirement
NFR-01	Performance	Page load time shall not exceed 3 seconds on standard broadband
NFR-02	Performance	API response time shall not exceed 500ms for 95th percentile requests

ID	Category	Requirement
NFR-03	Performance	WebSocket message delivery latency shall not exceed 1 second
NFR-04	Scalability	System shall support 500+ concurrent users
NFR-05	Availability	System shall maintain 99.5% uptime during operational hours (7 AM - 11 PM)
NFR-06	Security	All passwords shall be hashed with bcrypt (cost factor ≥ 10)
NFR-07	Security	JWT access tokens shall expire within 15-30 minutes
NFR-08	Security	System shall resist OWASP Top 10 vulnerabilities
NFR-09	Usability	UI shall be responsive across 7 breakpoints (475px to 1920px+)
NFR-10	Usability	System shall provide skeleton loading for all data-fetching states
NFR-11	Usability	All interactive elements shall have smooth animations (Framer Motion)
NFR-12	Maintainability	Codebase shall follow MVC architecture with clear separation of concerns
NFR-13	Portability	Frontend shall work on Chrome 90+, Firefox 88+, Safari 14+, Edge 90+
NFR-14	Reliability	Failed WebSocket connections shall auto-reconnect with exponential backoff

4.3 REST API Specification

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user	Public
POST	/api/auth/login	Authenticate user	Public
POST	/api/auth/refresh	Refresh access token	Public

Method	Endpoint	Description	Auth
POST	/api/auth/logout	Revoke refresh token	Authenticated
POST	/api/auth/google/callback	Google OAuth callback	Public
GET	/api/auth/me	Get current user	Authenticated
GET	/api/auth/verify-email	Verify email token	Public
POST	/api/auth/forgot-password	Request password reset	Public
POST	/api/auth/reset-password	Reset password with token	Public
GET	/api/users/profile	Get user profile	Authenticated
PUT	/api/users/profile	Update user profile	Authenticated
POST	/api/users/profile/image	Upload profile image	Authenticated
GET	/api/users/profile/image/:id	Get profile image	Public
PUT	/api/users/address	Update user address	Authenticated
GET	/api/canteens	List all canteens	Public
GET	/api/canteens/open	List open canteens	Public
GET	/api/canteens/:id	Get canteen details	Public
POST	/api/canteens	Create canteen	CANTEEN_OWNER
PUT	/api/canteens/:id	Update canteen	CANTEEN_OWNER
GET	/api/canteens/:id/menu	Get canteen menu	Public
POST	/api/canteens/:id/menu	Add menu item	CANTEEN_OWNER
PUT	/api/canteens/menu/:id	Update menu item	CANTEEN_OWNER
DELETE	/api/canteens/menu/:id	Delete menu item	CANTEEN_OWNER
GET	/api/cart	Get user cart	Authenticated
POST	/api/cart/add	Add item to cart	Authenticated
PUT	/api/cart/update/:id	Update cart item quantity	Authenticated
DELETE	/api/cart/remove/:id	Remove from cart	Authenticated

Method	Endpoint	Description	Auth
DELETE	/api/cart/clear	Clear entire cart	Authenticated
POST	/api/orders	Create order	Authenticated
GET	/api/orders/my-orders	Get user's orders	Authenticated
PUT	/api/orders/:id/status	Update order status	CANTEEN_OWNER
POST	/api/orders/:id/cancel	Cancel order	Authenticated
GET	/api/analytics/dashboard	Dashboard statistics	CANTEEN_OWNER
GET	/api/analytics/sales	Sales trend data	CANTEEN_OWNER
GET	/api/analytics/top-items	Top selling items	CANTEEN_OWNER

4.4 Database Schema

Entity	Key Fields	Relationships
users	id, email, password, fullName, mobile, role, authProvider, isActive, lockedUntil, mfaEnabled, profileImageData	1:N → orders, carts, reviews
canteens	id, name, location, description, isOpen, rushHourEnabled, fssaiNumber, gstNo, bankDetails, ownerId	1:N → menu_items, orders, coupons; N:1 → users
menu_items	id, name, description, price, category, isAvailable, isVeg, preparationTime, hasVariants, hasAddons, canteenId	N:1 → canteens; 1:N → variants, addon_groups, cart_items
menu_item_variants	id, name, price, menuItemId	N:1 → menu_items
addon_groups	id, name, minSelection, maxSelection, menuItemId	N:1 → menu_items; 1:N → addon_options
orders	id, orderNumber, customerId, canteenId, status, totalAmount, paymentMethod, paymentStatus, specialInstructions	N:1 → users, canteens; 1:N → order_items
order_items	id, orderId, menuItemId, quantity, price, variant, addons	N:1 → orders, menu_items

Entity	Key Fields	Relationships
carts	id, userId, canteenId, totalAmount	N:1 → users; 1:N → cart_items
cart_items	id, cartId, menuItemId, quantity, variant, addons, subtotal	N:1 → carts, menu_items
coupons	id, code, title, discountType, discountValue, scope, type (DISCOUNT/BOGO), canteenId, validFrom, validUntil	N:1 → canteens
reviews	id, orderId, customerId, canteenId, rating, foodRating, packingRating, deliveryRating, comment, vendorReply	N:1 → orders, users, canteens
categories	id, name, canteenId	N:1 → canteens
refresh_tokens	id, token, userId, expiresAt, revoked	N:1 → users
password_reset_tokens	id, token, userId, expiresAt, used	N:1 → users
password_history	id, userId, passwordHash, createdAt	N:1 → users
login_attempts	id, email, ip, success, timestamp	Audit log

5. Team Composition & Work Distribution

5.1 Team Members

#	Name	Role	Responsibilities
1	Kush Shah	Team Lead + Full Stack Developer + Security Tester	Project management, full-stack development, architecture design, security testing, code reviews
2	Krina Parikh	Frontend Developer	UI/UX implementation, responsive design, React component development, frontend state management
3	Ishan Shastri	Full Stack Developer	Feature development across frontend and backend, API integration, database design
4	Dhairi Tanna	Backend Developer + Security Architecture	Backend API development, security architecture, database optimization, authentication system
5	Shrey Raval	System/Security Tester + QA	Test case design, security testing, quality assurance, performance testing, bug tracking

5.2 Module-Wise Work Distribution

Module	Primary Owner	Support	Key Deliverables
Authentication & Security	Dhairi Tanna	Kush Shah	JWT auth, Google OAuth, CAPTCHA, account lockout, password reset, security headers
Frontend Architecture	Krina Parikh	Kush Shah	React app structure, routing, responsive layout, Tailwind config, animations
Student-Facing Pages	Krina Parikh	Ishan Shastri	Dashboard, menu browsing, cart, checkout, order history, profile
Vendor Dashboard & Analytics	Ishan Shastri	Krina Parikh	Order management, analytics charts, menu management, coupon UI

Module	Primary Owner	Support	Key Deliverables
Backend API & Services	Kush Shah	Dhairy Tanna	Controllers, services, repositories, JDBC queries, business logic
Database Design	Dhairy Tanna	Ishan Shastri	Schema design, migrations, indexes, seed data
WebSocket & Real-time	Kush Shah	Ishan Shastri	STOMP config, WebSocket service, frontend hook, live notifications
Coupon & Review System	Ishan Shastri	Dhairy Tanna	Coupon CRUD, discount logic, review submission, vendor replies
Testing & QA	Shrey Raval	All members	Test case design, manual testing, security testing, bug reporting
Documentation	Kush Shah	Shrey Raval	SRS, README, API documentation, code comments

6. Project Planning & Phases

6.1 Development Methodology

The project follows an **Agile-Waterfall Hybrid** approach — structured phases with iterative development within each phase. Each phase has defined deliverables and review checkpoints.

6.2 Phase-Wise Planning

Phase	Duration	Activities	Deliverables	Team Focus
Phase 1: Requirement Analysis & Planning	Week 1- 2	• Stakeholder interviews and requirement gathering • Feasibility study and technology evaluation • SRS document creation • UI/UX wireframe design • Database schema design	SRS Document, Wireframes, ER Diagram, Project Plan	Kush Shah (Lead), Shrey Raval (Requirements), All (Review)
Phase 2: System Design & Architecture	Week 3- 4	• High-level system architecture design • API contract definition (REST endpoints) • Database schema finalization • Security architecture planning • Frontend component	Architecture Document, API Spec, DB Schema, Security Plan	Dhairy Tanna (Security), Kush Shah (Architecture), Krina Parikh (UI Design)

Phase	Duration	Activities	Deliverables	Team Focus
		hierarchy design		
Phase 3: Core Backend Development	Week 5-8	• Spring Boot project setup and configuration • Database tables creation and JDBC repositories • Authentication system (JWT, OAuth, CAPTCHA) • Core services: User, Canteen, Menu, Category • Security configuration and headers	Working backend with auth, user, canteen, and menu APIs	Kush Shah, Dhairy Tanna, Ishan Shastri
Phase 4: Core Frontend Development	Week 5-8 (Parallel)	• React + Vite project setup with TypeScript • Design system: Tailwind config, colors, animations • Authentication pages (Login, Signup, Password Reset) • Student dashboard and canteen browsing • Menu browsing and cart implementation	Working frontend with auth flow, dashboard, menu, and cart	Krina Parikh, Ishan Shastri, Kush Shah

Phase	Duration	Activities	Deliverables	Team Focus
Phase 5: Order & Payment System	Week 9-11	• Order controller and service implementation • Checkout flow and order placement • Order management dashboard for vendors • Order history page for users • Payment method integration (COD initially)	Complete order lifecycle, vendor order management	Ishan Shastri, Kush Shah, Krina Parikh
Phase 6: Advanced Features	Week 12-14	• WebSocket integration (STOMP over WS) • Real-time order notifications • Coupon system (BOGO, percentage, flat) • Review and rating system • Analytics dashboard with charts • Profile image upload	Real-time notifications, coupons, reviews, analytics	Kush Shah (WebSocket), Ishan Shastri (Coupons/Reviews), Krina Parikh (Analytics UI)
Phase 7: UI Polish & Responsiveness	Week 15-16	• Responsive breakpoint strategy (xs to tv) • Skeleton loading for all pages • Framer Motion animations	Fully responsive UI, polished UX across all devices	Krina Parikh, Kush Shah, Shrey Raval (Testing)

Phase	Duration	Activities	Deliverables	Team Focus
		• Accessibility improvements • Cross-browser testing		
Phase 8: Security Hardening & Testing	Week 17-18	• OWASP vulnerability assessment • Rate limiting implementation • Input sanitization • Password policy enforcement • Security headers verification • Penetration testing • Performance testing	Security audit report, test results, bug fixes	Shrey Raval, Dhairy Tanna, Kush Shah
Phase 9: Integration Testing & Deployment	Week 19-20	• End-to-end integration testing • User acceptance testing (UAT) • Bug fixing and optimization • Production deployment preparation • Final documentation and README • Project presentation preparation	Deployed application, final documentation, presentation	All members, Shrey Raval (QA Lead)

6.3 Gantt Chart (Visual Timeline)

Phase	W1- 2	W3- 4	W5- 6	W7- 8	W9- 10	W11- 12	W13- 14	W15- 16	W17- 18	W19- 20
1. Requirements										
2. Design										
3. Backend Core										
4. Frontend Core										
5. Orders & Payment										
6. Advanced Features										
7. UI Polish										
8. Security & Testing										
9. Integration & Deploy										
Planning	Execution	Testing	Deployment							

6.4 Risk Management

Risk	Probability	Impact	Mitigation Strategy
WebSocket connection instability	Medium	High	Auto-reconnect with exponential backoff, SockJS fallback
Database performance bottleneck	Low	High	Query optimization, indexing strategy, connection pooling
Security vulnerabilities	Medium	Critical	OWASP compliance, regular security audits, penetration testing
Scope creep	High	Medium	Strict phase-based deliverables, backlog prioritization
Team member unavailability	Medium	Medium	Cross-training, documented codebase, shared knowledge

7. Appendices

Appendix A: Technology Stack Summary

Category	Technology	Purpose
Frontend Framework	React 18 + TypeScript 5	Component-based UI development
Styling	Tailwind CSS 3 + Custom Config	Utility-first responsive styling
Animations	Framer Motion	Page transitions and micro-interactions
Icons	Lucide React + Custom SVGs	Iconography
HTTP Client	Fetch API	REST API communication
WebSocket Client	@stomp/stompjs	Real-time STOMP messaging
Loading States	react-loading-skeleton	Skeleton placeholder UI
Backend Framework	Spring Boot 3.x	REST API server
Language	Java 21	Backend programming language
Database Access	Spring JDBC Template	SQL query execution
Security	Spring Security 6	Authentication and authorization
Token Management	jjwt (JSON Web Token)	JWT generation and validation
Encryption	BCrypt	Password hashing
Email	JavaMail + Brevo SMTP	Transactional emails
Database	PostgreSQL 15	Relational data storage
Build (Frontend)	Vite 6	Fast dev server and bundler
Build (Backend)	Maven	Java dependency management and build

Appendix B: Responsive Breakpoint Configuration

Breakpoint	Width	Columns (Grid)	Target Device
xs	475px	1	Small phones
sm	640px	1-2	Phones
md	768px	2	Tablets
lg	1024px	3	Small laptops
xl	1280px	4	Desktops
kiosk	1440px	4-5	Kiosk displays
tv	1920px	5-6	TV / large displays

Appendix C: WebSocket Topic Architecture

Topic	Direction	Payload	Subscriber
/topic/orders	Server → Client	Order object (JSON)	All vendors (global)
/topic/canteen/{'id'}/orders	Server → Client	Order object for specific canteen	Canteen vendor dashboard
/topic/canteen/{'id'}/order-updates	Server → Client	Updated order with new status	Canteen vendor dashboard
/topic/user/{'id'}/orders	Server → Client	User's order status change	User order history page
/topic/order-updates	Server → Client	Order status update (global)	All connected clients

Appendix D: Document Revision History

Version	Date	Author	Changes
1.0	February 2026	Kush Shah	Initial SRS document creation with all sections